

4.6 - OAuth 2.1 e OpenID Connect

Como identidade circula entre sistemas que não se conhecem

OAuth resolve delegação, não login

Acesso sem compartilhar a senha do usuário

- **OAuth autoriza acesso a recurso sem compartilhar senha**
 - O app recebe um token, nunca a credencial do usuário
- **Quatro papéis: resource owner, client, auth server, resource**
 - Cada um com responsabilidade clara no fluxo
- **OAuth sozinho não diz quem o usuário é**
 - Foi desenhado para autorização, não para autenticação
- **OpenID Connect adiciona a camada de identidade que falta**

Authorization Code com PKCE

- **PKCE adiciona um segredo dinâmico por requisição**
- **Protege apps públicos como SPA e mobile**

No OAuth 2.1, Authorization Code com PKCE é o fluxo recomendado para quase todos os casos. Implicit e password grant foram aposentados por inseguros.

- Client credentials continua válido para máquina a máquina

Fluxo Authorization Code com PKCE

PSEUDOCODE

1. Cliente gera verifier e challenge

```
verifier = random_string()
```

```
challenge = SHA256(verifier)
```

2. Redireciona para o auth server

```
GET /authorize?client_id=...&code_challenge=challenge
```

3. Usuario autentica e recebe um authorization code

```
=> redirect de volta com ?code=AUTH_CODE
```

4. Cliente troca o code por token, provando o verifier

```
POST /token { code: AUTH_CODE, code_verifier: verifier }
```

```
=> { access_token, id_token, refresh_token }
```

Login com as credenciais do cliente

- **Cliente enterprise quer login com Azure AD corporativo**
 - Funcionários não devem criar senha nova na LunaLog

LUNALOG

O cliente enterprise pediu que os funcionários dele fizessem login na LunaLog usando as próprias credenciais corporativas do Azure AD, sem criar senha nova nem memorizar mais um acesso. A LunaLog precisou implementar federação de identidade via OIDC pela primeira vez. O fluxo de Authorization Code com PKCE conectou o identity provider do cliente ao sistema da LunaLog sem que a equipe de Ana jamais tocasse nas credenciais corporativas. A LunaLog passou a confiar no token assinado pelo Azure AD, não na senha. Foi a primeira vez que a empresa delegou identidade em vez de gerenciá-la.

OIDC: identidade em cima do OAuth

- **OIDC adiciona o id_token ao fluxo do OAuth**
 - id_token é um JWT com claims de identidade
- **O claim sub identifica o usuário de forma estável**
 - Email, nome e grupos chegam de forma padronizada
- **Userinfo endpoint complementa os dados do usuário**
 - Discovery automatiza a configuração via well-known
- **OIDC é o que viabiliza SSO federado entre organizações**



OAuth e OIDC estabelecem como a identidade circula. O JWT é o formato que carrega essa identidade de um sistema para outro. Acontece que ele é, ao mesmo tempo, um dos padrões mais usados e mais mal implementados da web. Quais são as armadilhas que transformam um token confiável numa vulnerabilidade ativa?

JWT: estrutura, validação e armadilhas